UC San Diego

DSC 102
Systems for Scalable Analytics

Alexander Kolovos

Topic 1: Basics of Machine Resources
Part 1: Computer Organization

Ch. 1, 2.1-2.3, 2.12, 4.1, and 5.1-5.5 of CompOrg Book

DSC 102
Systems for Scalable Analytics

Logistics:

- Remember to register your PAs team Fri Apr 10!
- Instructor OH Zoom (Thu 5-6 pm)

Topic 1 – Outline

- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems
  - Process Management: Virtualization; Concurrency
  - Filesystem and Data Files
  - Main Memory Management
- Persistent Data Storage

The "Stored Program" Duality

The "Stored Program" epiphany highlights that bit sequences on a silicon chip can represent either

Data (e.g., a 16-bit integer) or

Software (e.g., an ARM machine code instruction)

Example: In the context of the Von Neumann model

Interpretation of stored values depends entirely on CPU-generated control signals.

- When the Program Counter (Instruction Pointer) points to a memory address, the Control Unit fetches the bit pattern at that location and treats it as an instruction to be decoded and executed.
- If an instruction (like a LOAD) points to that same address as an operand, the same bit pattern is retrieved and treated as data by the Arithmetic Logic Unit (ALU).

C Architectural Models And Parallelism

As a data scientist, you often work with "embarrassingly parallel" tasks. If you were designing a hardware accelerator specifically for a constant stream of independent matrix multiplications, which arch model be more efficient? Control-Flow (Von Neumann) or a Data-Flow approach?

In a Von Neumann system

- an Instruction Pointer must be constantly updated and managed for every single step of the matrix math, which creates a sequential bottleneck.

In a Data-Flow hardware accelerator (like a TPU)

- Hardware can be wired so that the results of one multiplication flow directly into the next addition as soon as they are ready, processing massive amounts of data in parallel without the overhead of tracking an "order" of instructions.

Think: Clock Speeds, Execution Time

A processor runs at **3 GHz**. A program has following instruction mix:

- 40% arithmetic instructions, **1 cycle each**
- 30% memory instructions, **5 cycles each**
- 30% control instructions, **2 cycles each**

Total instructions executed = $\mathbf{2 \times 10^9}$

Questions

1. What is the **average CPI (Clock cycles per instruction)?**
2. What is the **total number of cycles**?
3. What is the **execution time**?
4. If memory instructions increase to **50%**, with arithmetic dropping to **20%** and control staying at **30%**, compute new execution time.

Think: Memory Addressability

In a ARM-like system:

- Each instruction is **4 bytes**
- Instructions must be **4-byte aligned**

PC starts at **0x1000**

Questions

1. What are the next 5 instruction addresses?
2. How many instructions fit in **1 KB** of instruction memory?
3. What fraction of all 32-bit addresses are valid instruction addresses?

Think: Memory Dominates Performance

A program executes:

- **1 billion instructions**
- 30% are memory operations
- Each memory operation stalls for **100 cycles**
- CPU clock = **2.5 GHz**
- Non-memory instructions take **1 cycle**
- Assume memory instructions also take **1 base cycle** plus the stall

Questions

1. How many cycles are spent on base instruction execution?
2. How many cycles are spent waiting on memory?
3. What is the total execution time?
4. What percentage of total cycles are memory stall cycles?
5. If cache reduces stall from 100 cycles to 20 cycles, compute speedup.

Q: *What is data? How should we represent it?*

It may seem bizarre, but control of this is the basis for most computing today.

- ✓ Their **charge** lets us **manipulate** *electrons* easily.
- ✓ Their miniscule **size** and **mass** lets even a tiny amount of **energy** send them through wires and transistors at **the speed of light**, enabling small and fast, (i.e., desirable) computers.

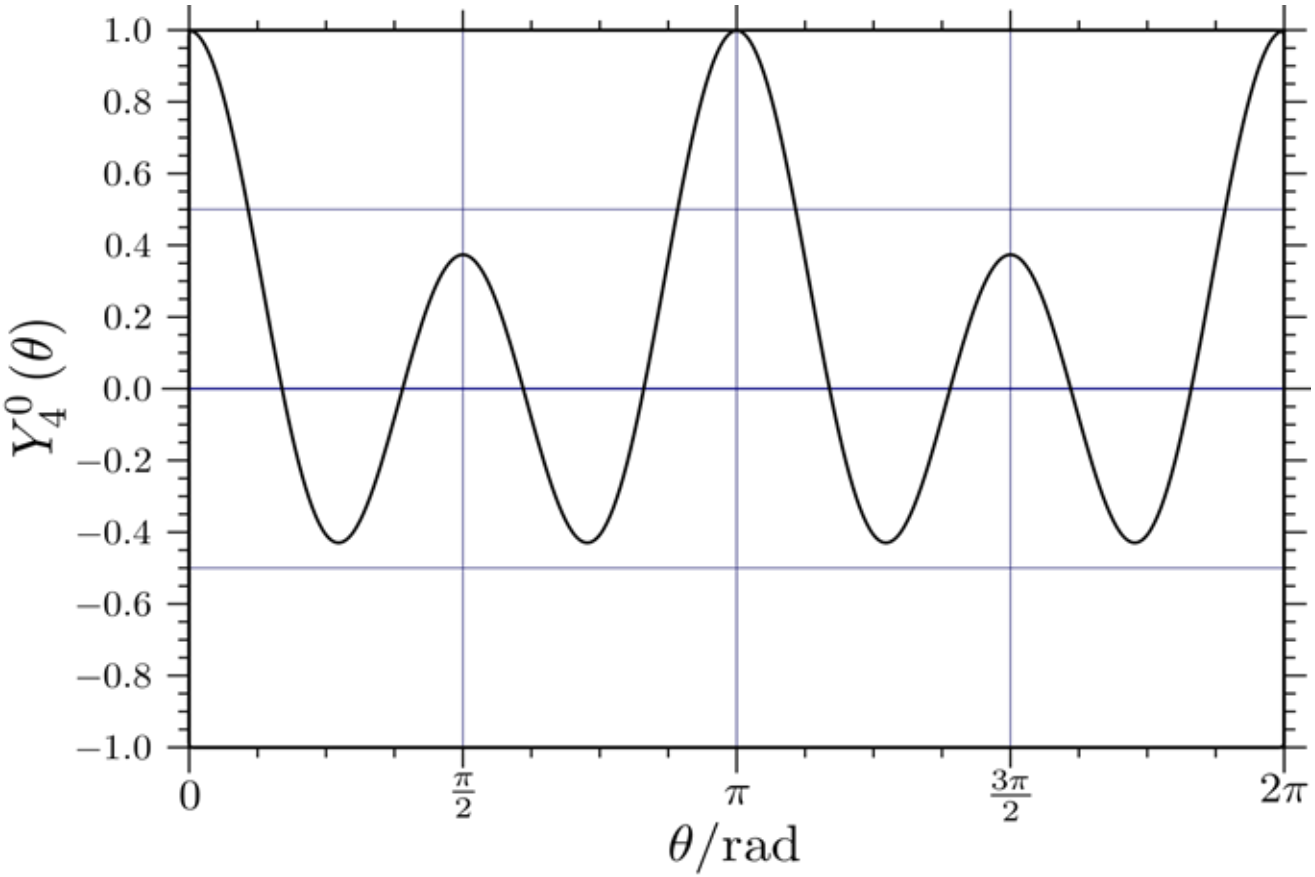
Automating Computation

- **Key insight (Alan Turing)**: Automating computation means building a machine to carry out a definite method on *symbols*.

- What can we use for symbols?
  - For arithmetic Real Numbers are attractive, but are **infinite in number** and of **infinite precision**.
  - 0.5000000000 = 0.4999999999 for made things.
  - We **cannot** manufacture a machine that has infinite capacity and infinite precision.

Analog World

- Analog is the domain of continuous values of arbitrary and finite precision.



The Game Of Telephone

The Scenario: Imagine you must pass the value of Pi to a classmate by drawing a line of that exact length (in cm) on a piece of paper.

The Task: Your neighbor must measure your line and draw their own version to pass to the next person.

Question: What do you think might happen?

What If We Try For An Analog Machine

- There will be errors in every value representation.
  For example:

  $$A + B = C$$

  becomes

  $$A + \epsilon + B + \epsilon' = C + \epsilon + \epsilon'$$

- Errors accumulate in the result with each successive computational step.
  - Like a game of telephone, results will quickly become *noisy.*

Digital Machine

- What if we restrict the machine to a **finite set of symbols**?
  - Feasible to build.
  - Strongly reduces the potential for error.

A Unary (Base-1) System

The Problem: An alien civilization uses a **Unary** system (tally marks) to track their population of 10 billion citizens.

The Calculation: If it takes 1 second to etch one tally mark, how long would it take to write down the current population?

The Comparison: How long does it take you to write a billion in Base-10?

Feel the pain of O(N)!

Symbol Set Examples

- Unary (base-1):

  The simplest set with only one symbol used for tallying.
- To represent the number **N**, write the (arbitrarily chosen) symbol **N** times. Example:
  - Let **A** be the number

    • • • • • •
  - Let **B** be

    • • • • • • • • • • • • • • • • • •
  - Then, **A + B** is

    •
- Unary addition requires concatenation, so it takes O(N) time to add a pair of numbers
  - OUCH! Too slow.

Minor Improvement To Unary

- Roman numerals:

  Add a few symbols

  I, V, X, L, C, M

  and some funky grammar rules to bunch together strings of ● (or strings of I, if you prefer) so larger numbers are less of a mess to write.
- Works *OK* for writing years in the Gregorian calendar

  ↓ ...But not so much for larger numbers.

  ↓ Plus, addition is hard and multiplication a nightmare.
- So, the underlying problem with unary persists.

Next Symbol Set

- Decimal (base 10):

  The ten symbols are
  0, 1, 2, 3, 4, 5, 6, 7, 8, 9

  ↓ Crucial idea: Weighted positional notation

  – N written as

  $$d_i \times 10^i + \ldots + d_1 \times 10^1 + d_0 \times 10^0 + d_{-1} \times 10^{-1} + \ldots + d_{-j} \times 10^{-j}$$

  10^i-place ... 10s-place 1s-place <--- Fractional part --->

  ↑ ↑

  Most significant digit Least significant digit

- N written in O(logN) symbols; MUCH shorter than Unary.
- Addition can be done in constant time.

A Mechanical Calculator

- A mechanism representing ten distinct symbols is easy enough.

- But there is a minimum precision for operation.
- Say ± 0.02 mm (or ±0.001 in)

Representing The Symbols

Observe that to represent ten symbols:

- Electrically, we would need
  - 10 distinct voltage levels.
  - allowance for imprecision of the electrical parts:

    Typical electrical part has ±5% variance from nominal specification.
- We also want FAST settling-in on one of 10 possibilities.
  - ! Settling in on fewer possibilities will be faster.

Think: Base-3?...

A startup claims they can make computers 50% faster by using **Base-3** (0V, 2.5V, and 5V) instead of Binary (0V and 5V).

Question:

Which system is more likely to mistake a "1" for a "2"?

Why did the industry settle on the "boring" Binary system instead of more "expressive" bases?.

Finally

The preceding discussion suggests we could ideally use

- Binary (base 2):

  A two-symbol, logarithmic representation achieves:

  ↓ (two-symbol) Electrical circuit need only move between and settle into one of two different voltage levels: FASTEST choice.

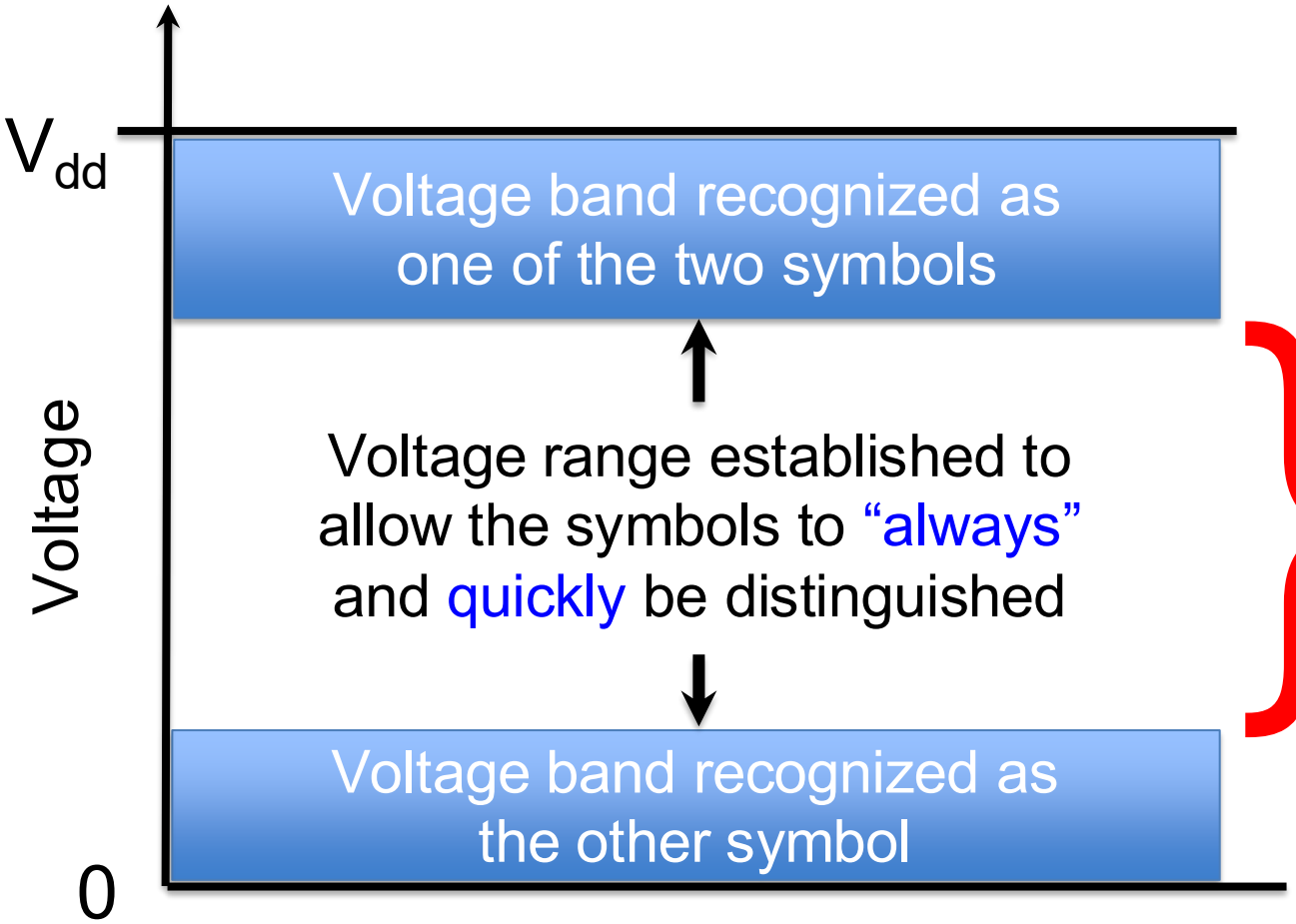
  ↓ (logarithmic) Avoiding the time complexity of unary.

- Representation must be tolerant of imprecision:

  ↓ Distinguishing symbols requires that the voltages be sufficiently different.

  ↓ Circuit imprecision requires acceptance of a range of voltages for each symbol.

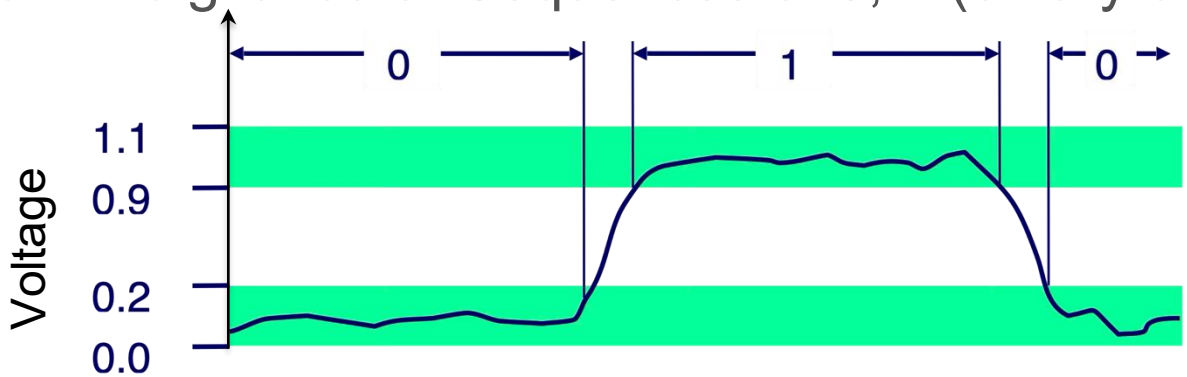
Electric Binary Symbol Representation



Crucial idea: This "gap" is how analog devices form a circuit that is "digital".

Digital Representation Building Blocks

- **Bits**: All digital data: Sequences of 0, 1 (binary digits). As we saw:



  - Physics enables electric implementation.
  - Very expressive and efficient.
  - Easy to store with bistable elements

    (e.g., by using high-low/off-on electromagnetism on disk.)
  - Reliably transmitted on noisy and inaccurate wires.
- **Qbits**: More (and different!) physics behind **quantum computing**: electron spin up (1), spin down (0) ...and every state in between!
  - Present course focuses on classical computing and using bits...

Some More Two-Symbol Systems

- Heads – tails

- Modulo 2 arithmetic $1 + 1 = 0$
- Statements – sentences that are either

True/False ambigram © Scott Kim

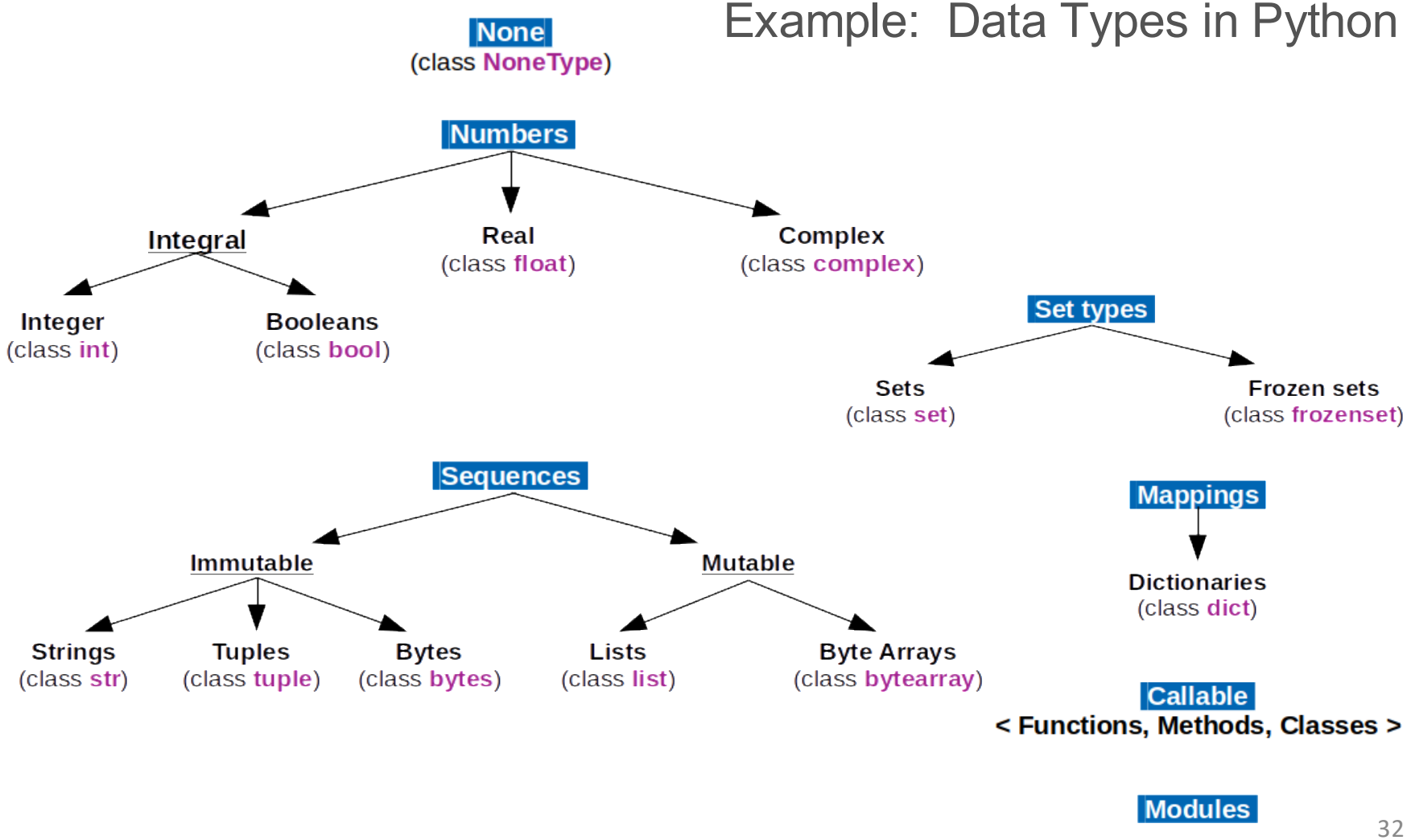
Digital Representation of Data

To facilitate interpretation of bit sequences, the following layers of *abstraction* are commonly used:

- **Data type:** First layer of abstraction to interpret a bit sequence with a human-understandable category of information; interpretation fixed by the programming language (PL).
  - Example common datatypes: Boolean, Byte, Integer, “floating point” number (Float), Character, and String
- **Data structure:** A second layer of abstraction to *organize* multiple instances of same or varied data types as a more complex object with specified properties
  - Examples: Array, Linked list, Tuple, Graph, etc.

Digital Representation of Data

Example: Data Types in Python



Digital Representation of Data

The *size* and *interpretation* of a data type depends on PL

A **Byte** (B; 8 bits) is typically the basic unit of data types.

Q: *How many unique data items can be represented by 3 bytes?*

- Given k bits, we can represent 2^k unique data items
- 3 bytes = 24 bits => 2^{24} items, i.e., 16,777,216 items
- Common approximation: 2^{10} (i.e., 1024) ~ 10^3 (i.e., 1000); conventions: kibibyte (KiB) is strictly 2^{10}; kilobyte (KB) might designate 2^{10} or 1000. When unsure, ask for clarification!

Q: *How many bits are needed to distinguish 97 data items?*

- For k unique items, invert the exponent to get $\log_2(k)$
- But "number of bits" is an integer! So, we only need $\lceil \log_2(k) \rceil$
- Therefore: We only need the next higher power of 2
- 97 ->128 = 2^7; so, 7 bits

Digital Representation of Data

Q: *How to convert from decimal to binary representation? (Approach A)*

1. Given decimal n

   if n is power of 2 (say, 2^k), put 1 at bit position k; if k=0, stop; else pad with trailing 0s till position 0

   if n is not power of 2, identify the power of 2 just below n (say, 2^k); #bits is then k; put 1 at position k
2. Reset n as n - 2^k; return to Steps 1-2
3. Fill remaining positions in between with 0s

	7	6	5	4	3	2	1	0	Position/Exponent of 2
Decimal	128	64	32	16	8	4	2	1	Power of 2
5_{10}						1	0	1	
47_{10}			1	0	1	1	1	1	
163_{10}	1	0	1	0	0	0	1	1	
16_{10}				1	0	0	0	0	

Digital Representation of Data

Q: *How to convert from decimal to binary representation? (Approach B)*

1. Given decimal n
   Repeat computation $\mathrm{mod}(n_i,2) = x_i$, where $n_0=n$ and $n_{i+1}=\lfloor n_i/2 \rfloor$
   Stop at the k-th iteration where $n_k=1$ and $\mathrm{mod}(n_k,2) = x_k = 1$.
2. The binary representation is the line-up of numbers $x_k\ x_{k-1} \ldots x_0$

	7	6	5	4	3	2	1	0	Position/Exponent of 2
Decimal	128	64	32	16	8	4	2	1	Power of 2
5_{10}						1	0	1	
47_{10}			1	0	1	1	1	1	
163_{10}	1	0	1	0	0	0	1	1	
16_{10}				1	0	0	0	0	

Q: *Binary to decimal?*

Digital Representation of Data

- Long bit strings can be unwieldy
  - 00010010101010000111110000011000
- Decimal notations do not fit the “power of 2” concept well.

To alleviate the above obstacles, a commonly used compromise is the Hexadecimal representation that uses Base 16.

  - Base 16 leads to much shorter strings.
  - Conversion between binary and Hex is easy.

Digital Representation of Data

- Hexadecimal representation is a common stand-in for binary representation; more succinct and readable
  - Base 16 instead of base 2 cuts display length by ~4x
  - Digits are 0, 1, ... 9, A (10_{10}), B, … F (15_{10})
- To convert a decimal to hexadecimal, it might be convenient to make a diversion by converting to binary first:
  - From binary, combine 4 bits at a time starting from lowest-value digits

Decimal	Binary	Hexadecimal
5_{10}	101_2	5_{16}
47_{10}	$10\ 1111_2$	$2F_{16}$
163_{10}	$1010\ 0011_2$	$A3_{16}$
16_{10}	$1\ 0000_2$	10_{16}

You might encounter equivalent alternative notations: $A3_{16}$, 0xA3 , $A3_H$

Data Types: Boolean And Integer

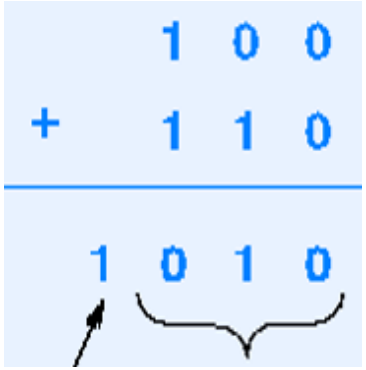
- **Boolean**:
  - Examples in data science: Y/N or T/F responses
  - Just 1 bit needed but actual size is almost always 1B, i.e., 7 bits are wasted! (***Q:*** *Why?*)
- **Integer**:
  - Examples in data science: #friends, age, #likes
  - Typically 4 bytes; many variants (short, unsigned, etc.)
  - Java *int* can represent -2^{31} to $(2^{31} - 1)$;
    (*Why? try simple example with 3 bits by hand to verify*)
  - C *unsigned int* can represent 0 to $(2^{32} - 1)$;
  - Python *int* is effectively unlimited length (PL magic!)

Data Types: Integer: Negative Values

- Positional interpretation (translating an entire sequence of bits to a number) produces unsigned integers.
  - No provision for negative values, but an unsigned integer representation can store integers that are twice as large as in a signed representation.
  - Arithmetic operations can produce overflow or underflow (result cannot be represented in k bits). This condition can be handled with the so-called carry bit.

Example: In the side plot, hardware records an overflow in a separate carry bit indicator.

- Software must test for the carry bit, if programmer wishes to proceed differently in overflow case.
- The overflow may also be used to raise an exception and guide hardware to a series of pre-defined steps, regardless the programmer's code.



Overflow Result

Data Types: Integer: Negative Values

- Signed arithmetic is needed by most programs, where some bit patterns are used for negative values.
  - Each signed integer representation has its own quirks. Some of those representations that have been used in systems include:
    - Sign magnitude
    - One's complement
    - Two's complement

Demonstration: Two's complement

- Positive number uses positional representation.
- Negative number is formed by subtracting 1 from positive value and inverting all bits of result.

Observe: The higher-order bit is set, if the number is negative.

Example: A 4-bit Two's complement:

0010 represents the decimal integer 2

1110 represents the decimal integer –2

Data Types: Integer: Negative Values

Demonstration: Unsigned vs. signed representations of a 4-bit integer

The table shows a listing of the binary values for all combinations of a 4-bit integer, and each sequence's corresponding decimals for the unsigned Two's complement interpretation.

Interesting observations about the decimals in the signed Two's complement interpretation:

- The negative decimals correspond to binary values with the higher-order bit set to 1.
- There is one more negative value than positive value.
- Zero (0) has only one representation.

Binary Value	Unsigned Interpretation	Two's Complement Interpretation
1111	15	−1
1110	14	−2
1101	13	−3
1100	12	−4
1011	11	−5
1010	10	−6
1001	9	−7
1000	8	−8
0111	7	7
0110	6	6
0101	5	5
0100	4	4
0011	3	3
0010	2	2
0001	1	1
0000	0	0

Data Types: Character

- Representing **Character (char)** and **String**:
  - A set of symbols that computers and I/O devices understand
  - Represents letters, numerals, punctuations, etc.
  - A string is typically just a variable-sized array of char
  - C *char* is 1 byte; Java char is 2 bytes; Python does not have a char type (use *str* or *bytes*)

An encoding is the correspondence of bit sequences to character / symbol sets.

Data Types: Character Old Encodings

Early computers had wildly incompatible sets. There were computers with as few as 6-bit characters and others with 16-bit characters. Different codes were developed to standardize encodings:

- EBCDIC: **E**xtended **B**inary **C**oded **D**ecimal **I**nterchange **C**ode
  - 8-bit encoding, defined by IBM. Popular in 1960s.
  - Still used on IBM mainframe computers
  - Example encoding: lower case letter ‘a’ assigned binary value
    - 10000001
- ASCII: **A**merican **S**tandard **C**ode for **I**nformation **I**nterchange
  - 8-bit encoding; prevailed prior to the broader adoption of Unicode.
  - Only 7 bits are used for a character; allows for encoding 2^7 = 128 characters.
  - ASCII covers all types of characters in English, but cannot accommodate all character sets from different languages and scripts; for example, ASCII is inadequate for representing the tens of thousands of characters in Chinese.

Data Types: Character – Unicode

The ASCII limited set of different characters led to significant issues across the world.

– Example: On some PCs, character code 130 would display as é, but on computers sold in Israel it was the Hebrew letter Gimel (ג). Consequently, when Americans would send résumés to Israel, they would arrive as rגsumגs.

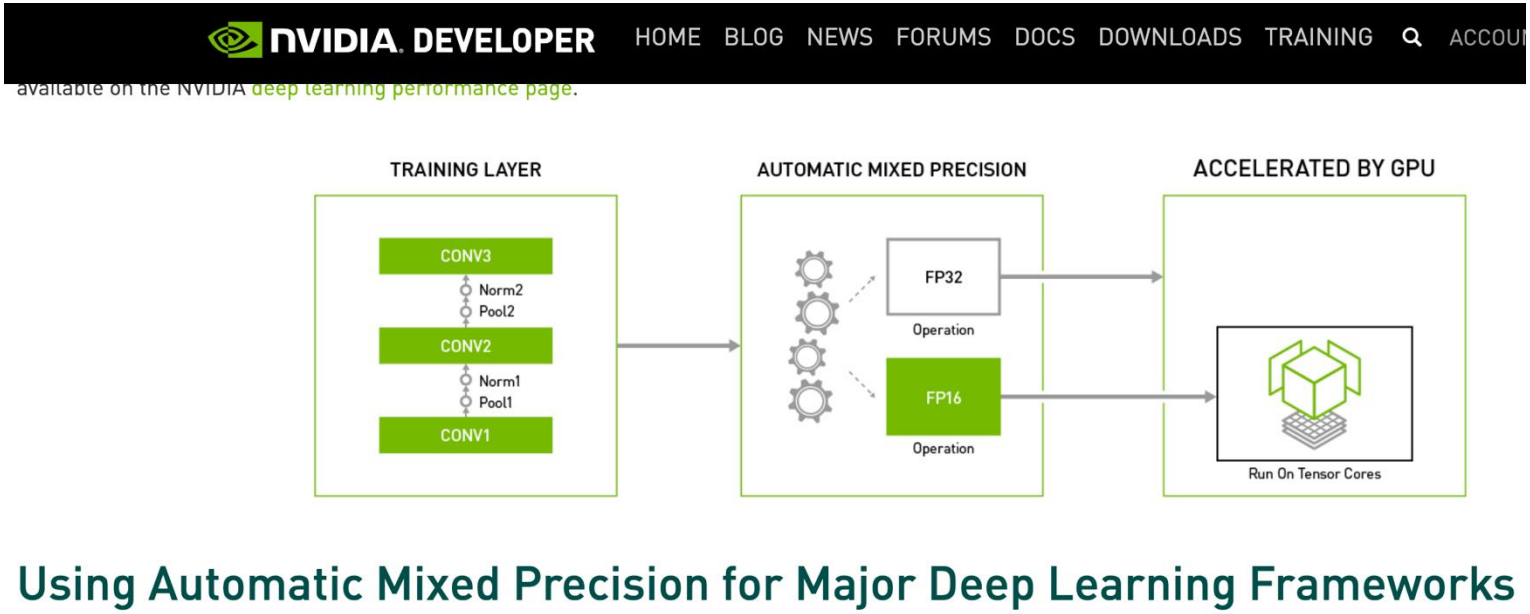
Unicode was a brave effort to *create a single character set* that included every reasonable writing system on the planet. Allocating additional bits per character enabled representing even make-believe character sets or emojis as bit sequences.

- Depending on the encoding, Unicode takes anywhere from 1-4 bytes, to create the repsective formats UTF-8 to UTF-32 (Universal Transformation Format).
- Unicode *disassociates the platonic characters from the byte representation via Code points.*
- Unicode maintains complete backward compatibility with ASCII.

All modern software conforms (or is supposed to) to Unicode for multi-lingual capability.

Data Types: Float

- **Float**:
  - Examples in data science: salary, scores, model weights
  - IEEE-754 single-precision format is 4B long
  - IEEE-754 double-precision format is 8B long
  - Java and C *float* is single; Python *float* is double!



Data Types: Float

In computers, floats are represented in binary scientific notation (IEEE-754 standard.)

The concept is identical to the decimal scientific notation. For example,
a number z = -0.752 can be equivalently written in scientific notation as $z = -7.52\times10^{-1}$
In the latter expression, we distinguish the following fields:

- Sign s: Indicates positive or negative number.
- Exponent (the power of 10) E: Determines number range.
- Mantissa (or Fraction) M: Significant digits; determines precision.
  Note: Left-most (most significant digit) is always non-zero.

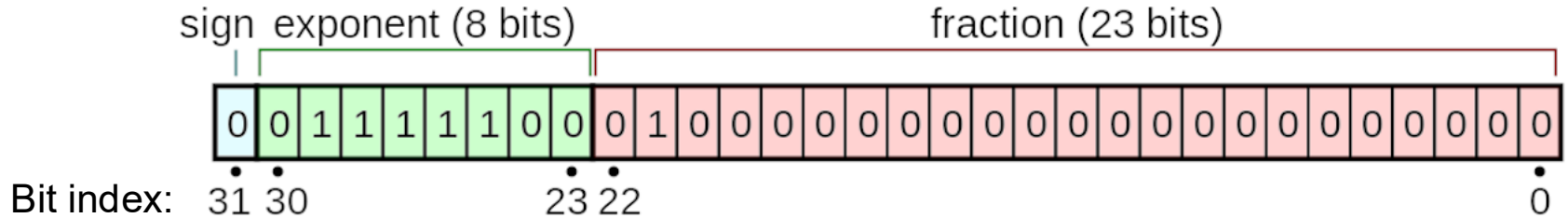
In these terms, the scientific notation of the example number z can be fully rewritten as
$z = -1^s \times 10^E \times (7\times10^0 + 5\times10^{-1} + 2\times10^{-2})$ where s=1, E=-1, and (in parenthesis) M=7.52.
More generally, all we need to determine a number z are s, E, and M.

- Specifically, a decimal float can be represented as a concatenation (•) z = s•E•M
  which can be interpreted as $z = -1^s \times 10^E \times \left(\sum_{i=1}^{N} d_i \times 10^{-i} \right)$, given N significant digits.

Computers represent floats in a near-identical manner, by using concatenated bits to designate the sign, exponent, and mantissa in the binary scientific notation of the float.
Next up, we examine this representation (and where it differs from decimals above).

Data Types: Float

- This is the IEEE standard float32 (4-bytes) representation.



- The 8-bit field of the exponent E ranges in [0, 255]. As is, it can only represent positive powers of 2. To amend this for the binary scientific notation, the IEEE-754 standard introduces, instead, the

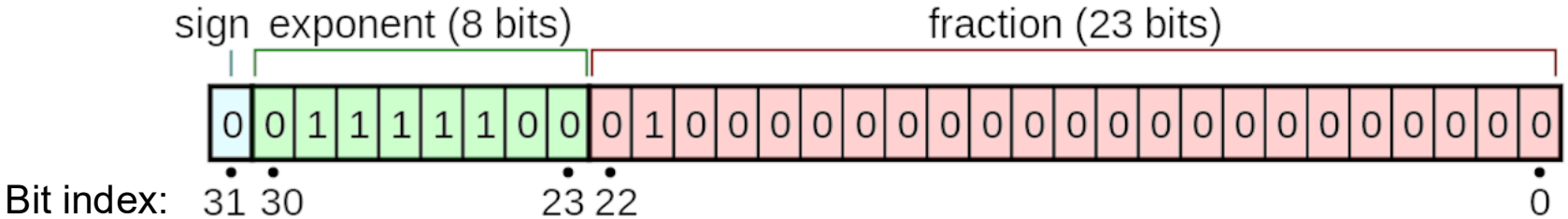
  Biased Exponent e = E - 127, so that e ranges in [-127, 127].

  Note: The value E=255 is reserved to designate special Infinite/NaN values.

- We noted that in the mantissa M, the left-most (most significant digit) is always non-zero. In the binary system, value 1 the only alternative for a non-zero bit, so mantissa will look like $1.b_{22}b_{21}\ldots b_1b_0$. The first bit 1 is one more bit of precision "for free", and it is commonly known as the hidden bit or implicit bit because
  - Neither floating point registers nor memory store the 1 in the beginning.
  - ALU inserts this 1 bit into each incoming operand, uses it within ALU circuits, then strips it from each result on the way to storage.

Data Types: Float

- This is the IEEE standard float32 (4-bytes) representation.



Due to the hidden bit, the decimal representation of mantissa's 23-bit string becomes $1 + (\sum_{i=1}^{23} b_{23\text{-}i} \times 2^{-i})$

Altogether, the preceding remarks make it straight-forward to derive the decimal interpretation of a 32-bit float: $z = -1^{s} \times 2^{E-127} \times [\, 1 + (\sum_{i=1}^{23} b_{23\text{-}i} \times 2^{-i}) \,]$ from its binary IEEE representation.

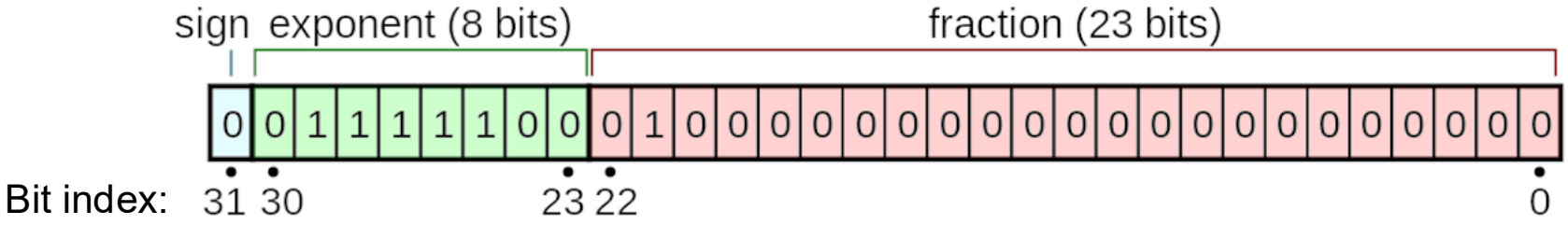
Example: Assume the 32-bit float z in the depicted s•E•M concatenation above. Then:

- sign bit s = 0, therefore $-1^{s} = -1^{0} = 1$
- exponent $E = (1111100)_2 = (124)_{10}$, therefore $2^{E-127} = 2^{124-127} = 2^{-3} = 0.125$
- mantissa $M = 1 + (\sum_{i=1}^{23} b_{23\text{-}i} \times 2^{-i}) = 1 + (1 \times 2^{-2}) = 1 + 0.25 = 1.25$

The decimal interpretation expression gives then: $z = 1 \times 0.125 \times 1.25 = 0.15625$

Data Types: Float

- This is the IEEE standard float32 (4-bytes) representation.



One more note:

In the Biased Exponent e = E-127, the bias=127 value that was introduced by the IEEE makes comparing two FP numbers much easier, as follows:

Consider FP numbers x_1 and x_2 and their corresponding concatenated bit strings $s_1 \bullet E_1 \bullet M_1$ and $s_2 \bullet E_2 \bullet M_2$ as sign magnitude numbers.

– FP numbers $x_1 > x_2$ **if and only if** $s_1 \bullet E_1 \bullet M_1 > s_2 \bullet E_2 \bullet M_2$

The above perspective enables us to compare FP numbers by using integer hardware.

(**NB:** Converting decimal reals/fractions to float32 is NOT in syllabus!)

Digital Representation of Data

- Due to representation imprecision issues, floating point arithmetic (addition and multiplication) is not associative!

```
(env311) akmbp:~ alexander$ python
Python 3.11.10 (main, Oct  3 2024, 02:37:52) [Clang 14.0.6 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 0.1 + 0.2
0.30000000000000004
>>> (0.1 + 0.2) + 0.7
1.0
>>> 0.1 + (0.2 + 0.7)
0.9999999999999999
>>>
```

- In binary32, special encodings recognized:
  - Exponent 0xFF and fraction 0 is +/- “Infinity”
  - Exponent 0xFF and fraction <> 0 is “NaN”
  - Max is ~ 3.4×10^{38}; min +ve is ~ 1.4×10^{-45}

Digital Representation of Data

Order in storing bits and bytes

The design needed for the physical layout of bits and bytes in storage

– affects connection (wires) between storage and ALU

– informs how to send/receive data via a network

The order of bits is handled by hardware, and is usually hidden from software.

However, memory is most often byte-addressed.

- How multi-byte items span bytes is a HW design choice
- I/O done via memory, so *byte order of multi-byte items may need translation between communicating computers*.

Byte order is important and should be visible to the programmer.

Digital Representation of Data

Order in storing bits and bytes

There are 2 main designs for storing bits and bytes:

- Big-endian stores the most significant byte (MSB) first (matches human left-to-right reading).
- Little-endian stores the least significant byte (LSB) first.

Example: For the 4-byte hex value 0x12345678

- using big-endian byte storage, number is stored as 12 34 56 78
- using little-endian byte storage, number is stored as 78 56 34 12

Digital Representation of Data

- More float standards: double-precision (float64; 8B) and half-precision (float16; 2B); different #bits for exponent, fraction
- Float16 is now common for *deep learning* parameters:
  - Native support in PyTorch, TensorFlow, etc.; APIs also exist for weight quantization/rounding post training
  - NVIDIA Deep Learning SDK support mixed-precision training; 2-3x speedup with similar accuracy!
- New processor hardware (FPGAs, ASICs, etc.) enable arbitrary precision, even 1-bit (!), but accuracy is lower

https://docs.nvidia.com/deeplearning/sdk/mixed-precision-training/index.html

Accuracy vs. Efficiency

The Dilemma: You are deploying a self-driving car AI. You can use **Float64** or **Float16**

The Question:

- If Float16 makes the car react 200ms faster but misidentifies a traffic cone as a pedestrian 0.1% more often, which do you choose?

- How does "Weight Quantization" (reducing bits) affect the battery life of a mobile phone running an LLM?.

.

Digital Representation of Data

- All digital objects are *collections* of basic data types (bytes, integers, floats, and characters)
  - SQL dates/timestamp: string (w/ known format)
  - ML feature vector: *array* of floats (w/ known length)
  - Neural network weights: *set* of multi-dimensional *arrays* (matrices or tensors) of floats (w/ known dimensions)
  - Graph: an *abstract data type* (ADT) with *set* of vertices (say, integers) and *set* of edges (*pair* of integers)
  - Program in PL, SQL query: string (w/ grammar)
  - DRAM addresses: *array* of bytes (w/ known length)
  - Instruction in machine code: *array* of bytes (w/ ISA)
  - Vector Store object: *array* of floats (to fit LLM embeddings; possibly accompanied by an additional integer to use as index)
  - Other data structures or digital objects?

Digital Representation of Data

- **Serialization** and **Deserialization:**
  - A data structure often needs to be persisted (stored in a file) or transmitted over a network
  - Serialization is the process of converting a data structure (or program objects in general) into a neat sequence of bytes that can be exactly recovered; deserialization is the reverse, i.e., bytes to data structure
  - Serializing bytes and characters/strings is trivial
  - 2 alternatives for serializing integers/floats:
    - As *byte stream* (aka “binary type” in SQL)
    - As *string*, e.g., 4B integer *5* -> 2B string as *“5”*
    - String serialization common in data science (CSV, etc.)

Review Questions

- What is the difference between data and code?
- What kind of software is TensorFlow? Linux?
- Why do computers use binary numbers?
- What is a byte?
- How many integers can you represent with 5 bits?
- How many bits do you need to represent 5 integers?
- What is the hexadecimal representation of 20_{10}?
- Why is a floating point standard needed?
- Why should a data scientist know about float formats?
- What does “lower precision” mean for a float weight in DL?
- Why is serialization needed on a computer?
- Is code a string? Is a string code?
- Is reality a computer simulation? :)

Appendix: Example Interpretation

A: Bit string in IEEE FP into decimal scientific notation.

Interpret the following bit string in IEEE FP, then write its value in decimal scientific notation

1100 0001 1111 0000 0000 0000 0000 0000

Answer:

Let us assume the above 4-byte FP as a bit sequence x = sEM where s is the sign field, E is the exponent field, and M is the Mantissa field. Then, it will be

sEM = 1 10000011 11100000000000000000000

Recalling hidden 1, IEEE FP fields literally say

– 131 1.11100000000000000000000

which is $-1.111 \times 2^{131-127} = -1.111 \times 2^4$

To write in decimal scientific, normalize to obtain exponent=0 so radix change is trivial ($2^0 = Y^0$ for any Y a positive integer). This yields finally:

$-1.111 \times 2^4 = -11110.0 \times 2^0 = -30.0 \times 10^0 = -3.0 \times 10^1$

Appendix: Example Interpretation

B: Decimal scientific notation into Bit string in IEEE FP.

What bit string represents -9.175×10^1 in IEEE FP?

Answer:

Step 1: Determine sign bit: Sign = –

Step 2: Express as rational number: $9.175 \times 10^1 = 91.75$

Step 3: Convert decimal rational number to binary: $(91.75)_{10} = (1011011.11)_2$

Step 4: Normalize binary rational number:

$1011011.11 = 1.01101111 \times 2^6$

Step 5: Compute E = e + Bias = 6+127 = 133 = $(10000101)_2$

Step 6: Strip mantissa of MSB digit 1, and assemble fields:

1 10000101 01101111000000000000000 (or 0xC2B78000)